Project Report

CSCI 5253: Data Center Scale Computing

1. **Title and list of project participants**

Title: PPTGaze - Presentation extraction service.

Team (Project Group 120):

1. Prathik Bharath Jain
2. Bhuvvaan Punukolu

2. **Video Recording and Git Repo**
    1. Video link: https://youtu.be/hMuxLK65RBs
    2. Git repo: https://github.com/bhuvvaan/dcsc-final-projet
3. **Project Goals**:

    PPTGaze is a service which takes presentation videos as input, extracts slide from the presentation and returns a presentation document. Generally, we miss the visual(pictorial) aid used in the presentation when summarizing the videos, which are used to explain the content better.

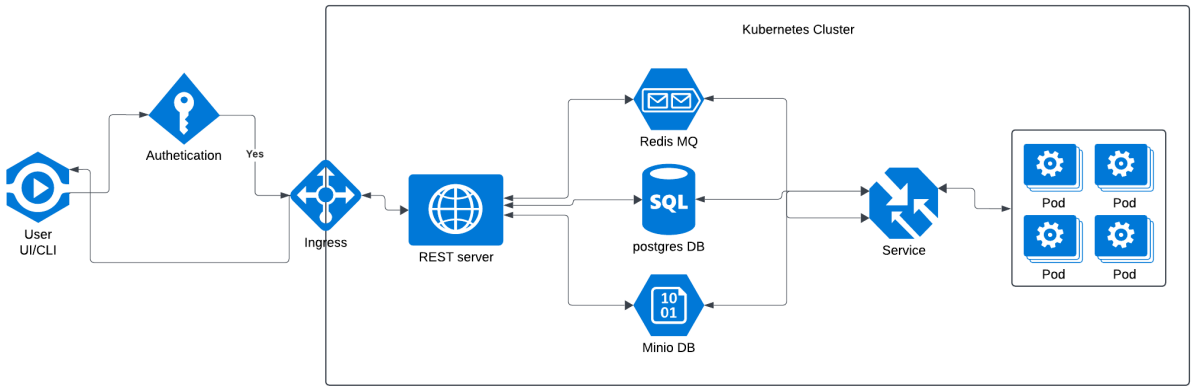
    We are building a service that captures the presentation from the video lectures and provide the summarized speakers notes to the presentation, which can save a lot of time without impacting the learning curve.

4. **A *specific* list of software and hardware components**

    Software Components:

    1. Compute Instance
    2. Kubernetes
    3. Postgres Database
    4. Message queue (Redis)
    5. Storage service: Minio
    6. Virtual Machine

5. **An *architectural diagram showing the interactions* between system components -** see this article for a reasonable example



6. **Value addition when compared to baseline (lab 7)**

    1. UI: A web interface using HTML/CSS/JavaScript will be created that will assist in easy login and upload to the user.
    2. Authentication: We will be using authentication DB to store email id and password and authenticate the user.
    3. Caching DB: A caching SQL DB is used that checks if the video uploaded already exists in the object DB and just returns the ppt link rather than running the pods again.
    4. Core brain algorithm: An algorithm that segments presentation from the video frame by frame and converts it into a ppt or pdf needs to be created from scratch.

7. **Explain the working system, what kind of workload can it handle, where are potential bottlenecks?**

    Users will first authenticate themselves via a password. Once they are authenticated, they will have an option to upload the video file via CLI or UI. The schema of the data will be the following:

    Input

User ID	Video Name	Video ID

The first two columns will be provided by the user, but the third column will be added by the program for identification purposes.

REST Server

The video file is then received by a REST server. The rest server takes the responsibility of posting in the message queue, uploading to the object DB and uploading the file metadata in a SQL DB.

MQ

We are using a Redis MQ. This queue stores messages from REST server that the file is successfully uploaded to object DB and is ready for processing. The interaction is two ways as this queue also stores messages from worker pods after the processing is complete. Input schema to MQ will be

User ID	Video ID	Video Link	Status

Status column will be different depending on who is writing to the queue.

SQL

A SQL DB will hold all the metadata related to our service. The rest server will update this after upload to object DB is complete. Workers also have read/write permissions to the DB.

User ID	Video ID	Video Link	Status	PPT link

The PPT link and status is updated by worker nodes after the processing is complete. This service will be used for caching purposes. When a user submits the same file multiple times, we just look in the SQL DB and return the already existing link.

Object DB:

The Object DB has write access from both REST server and worker nodes. It essentially stores the video files, ppt and their identifiers.

Worker Pods:

The worker pods take video files from the DB and process them. We will use Kubernetes deployments and services to effectively load balance the nodes.

Return to Rest:

The rest server picks the message from message queue and downloads the file in the Kubernetes cluster. This file is converted to json format and sent to the UI as a downloadable file.

Potential bottle necks:

1. While giving an id for a file we are only using a five-digit integer. This could lead to issues if the traffic increases as hashes collide.
2. We have not implemented autoscaling, which will lead to issues and denial of service if the traffic increases.
3. Large video files could cause failures.
4. The authentication is not completely safe as it is plaintext, and we are not implementing any password checks.

8. **A description of how you debugged your project and what training or testing mechanism were used.**

Training: We will be inputting multiple video files to train the model and find the best frequency threshold to capture the screen, this would play a major role as it directly impacts the processing time and the quality of the report.

Input : Video files , output : Frequency threshold for capturing frames

Another important part of the training would be set a similarity threshold to compare the frames captured as to decide to consider as a new slide or reiterate further.

Input: Video frames, output: Threshold for similarity

Testing: The most important and basic, functional testing (unit test) would be performed to analyze the complete working. Integration testing to verify the components are compatible with respect message formats and firewall access. We would also test the API performance by testing them via postman.

Debug: As we are using the message queue between the service for the interaction, we can refer the topics and trace the message history. Another way could be to ssh into the system and grep for the logs. We will be following the log levels of Error and Debug in the service and make sure to add in an identifier in all the error messages.